

pool_layout_v1 Rationale

Status: Companion document

Version: 1.0

Language: English

Audience: developers, packagers, operators, runtime integrators

Purpose

`pool_layout_v1` defines a shared filesystem layout for LLM artifacts on hosts that may run multiple inference engines. This companion document explains why the specification is format-first, why published artifacts are treated as immutable, why caches and staging areas are kept separate, and why symlink and hardlink behavior is constrained.

The layout is intended to reduce surprises for operators, lower integration cost for runtime authors, and prevent accidental corruption of shared model storage. It is also meant to be usable in both rootless and system-managed environments without changing the conceptual model.

Why the layout is format-first

The most important design decision in `pool_layout_v1` is that published artifacts are grouped first by artifact format rather than by runtime name.

This choice exists because artifact format is more stable than runtime attachment. A single GGUF model can be relevant to more than one runtime, especially within the `llama.cpp` family and its forks.[cite:73][cite:63] By contrast, runtime capabilities change over time, and the same runtime may support several formats across releases.

This is especially visible with GGUF. `llama.cpp` treats GGUF as a native artifact type, while `vLLM` documents GGUF support as highly experimental and currently limited enough that it should not define the canonical storage taxonomy.[cite:30][cite:24] If the pool were organized primarily as `vllm/`, `llama.cpp/`, or similar runtime buckets, the same artifact could end up duplicated or ambiguously assigned.

A format-first tree also makes future expansion cleaner. New runtime integrations can reuse the existing artifact families instead of forcing a storage migration. That lowers operational risk and lets the filesystem layout outlive individual runtime trends.

Why published artifacts are canonical and immutable

The specification treats `artifacts-root` as the only canonical location for published model payloads. This means operators, indexers, and runtimes have exactly one place to look for durable model files.

Immutability is required because model payloads are large, expensive to verify, and often shared across many consumers. A runtime that writes back into the published tree can silently invalidate checksums, break deduplication assumptions, or corrupt a model for every other consumer attached to the same host. In practice, runtimes often maintain their own writable caches or local download areas, which is another reason to keep writable activity outside the published artifact tree.[cite:30]

Immutability also improves reasoning. When a file appears under the canonical published path, other tools may safely assume it has already passed import-time validation, naming normalization, and publication rules. Without that property, the published tree stops being a trustworthy source of truth.

Why caches are separate

`cache-root` exists because runtime state is not the same thing as model content. Prompt caches, merged outputs, engine-specific accelerators, temporary download caches, and derived indexes may improve performance, but they are not the model itself.

Keeping cache data out of `artifacts-root` prevents two classes of failure. First, it avoids turning a clean model pool into a mixed directory of payloads plus runtime debris. Second, it makes deletion policy much safer, because caches can usually be removed and regenerated without redefining what the host actually owns.

This separation also matches the behavior of existing tooling. For example, the Hugging Face documentation for `llama.cpp` notes a dedicated `LLAMA_CACHE` environment variable for cache placement, which reinforces the idea that canonical model files and runtime cache material belong in different locations.[cite:30]

Operationally, a separate cache root helps container deployments. `artifacts-root` can be mounted read-only, while `cache-root` remains writable. That is a safer default for rootless systems and shared hosts because it sharply narrows the set of paths where mutation is allowed.

Why staging is separate

`staging-root` exists to absorb the messiness of real imports. Downloads can fail halfway through, uploaded files may be incomplete, conversion jobs may generate invalid output, and validation may reject the candidate model entirely.

If those incomplete artifacts land directly in the published tree, readers can observe partial state and mistake it for a usable model. Separating staging from publication creates a simple contract: everything in staging is provisional, and nothing there is yet part of the published pool.

This design also enables atomic publication. A tool can prepare and verify the full artifact set in staging, then publish it in one final step. That protects readers from partial updates and makes rollback straightforward: a failed import stays in staging and never contaminates the published namespace.

Why the path shape is `format/publisher/model/file`

The path shape chosen by `pool_layout_v1` captures identity at the same level users already expect from model ecosystems. Publisher or namespace comes first, then model identity, then the concrete artifact files.

This is a practical fit for ecosystems that already express models as publisher-or-organization plus repository name. The Hugging Face and LM Studio workflows both lean on that mental model, which makes `publisher/model` a natural directory boundary for operators and developers.[cite:38][cite:40] The result is easier manual browsing, cleaner indexing, and simpler import logic.

Grouping all files for one upstream identity in a single model directory also keeps related payloads together. For GGUF, that means multiple quantization files can live beside one another without inventing synthetic runtime directories. For safetensors-style deployments, configuration and tokenizer sidecars remain adjacent to the model weights.

Why the specification supports symlinks but restricts them

Symlinks are useful because they allow controlled aliasing and can reduce duplicate storage when several names intentionally refer to the same published payload. They are particularly convenient during migration or when implementing compatibility aliases.

Unrestricted symlinks, however, are dangerous in a shared pool. A symlink that points outside the trusted pool boundary can make readers consume arbitrary host files, staging leftovers, or runtime cache data. That defeats the central purpose of having a canonical published tree.

For that reason, the specification allows symlinks only under tight rules: they should resolve inside the trusted published boundary, and relative symlinks are preferred because they are more portable across mounts and host moves. File-level symlinks are safer than directory-level symlinks because they reduce the blast radius of mistakes.

Why the specification supports hardlinks but treats them carefully

Hardlinks are a practical optimization when importing large files on the same filesystem. They can make publication much faster and avoid needless data duplication.

The trade-off is that a hardlinked published file shares the same inode as its source. That is safe only if the implementation truly treats the published artifact as immutable after publication. Any later write to the source path could modify the published file as well.

That is why `pool_layout_v1` permits hardlinks for safe publication workflows but forbids using them to smuggle mutable or untrusted data into `artifacts-root`. Hardlinking from cache content is especially risky because caches are, by definition, writable and disposable.

Why import modes are explicit

The specification recognizes three publication styles: copy, hardlink, and symlink. These modes have different operational semantics, and hidden behavior creates confusion for operators.

An explicit import mode makes debugging easier. If a model was published by copy, it is independent. If it was published by hardlink, inode sharing matters. If it was published by symlink, path resolution and trust-boundary checks matter. Recording that mode in import metadata turns storage behavior into something inspectable instead of implicit.

The recommended default order, hardlink when safe and same-filesystem, otherwise copy, reflects a balance between efficiency and predictability. Symlink import is intentionally not the default because it is the easiest mode to misuse in shared environments.

Why the layout works well for rootless systems

A rootless deployment needs a layout that does not assume privileged writes to the published tree during ordinary runtime execution. The split between published artifacts, writable caches, and staging jobs supports that model directly.

In practice, a service account can own its writable cache and staging roots while still consuming a shared read-only artifact tree. This keeps the permission model clear: only trusted importers publish artifacts, while runtimes consume them and write their own transient state elsewhere.

That model also maps naturally to container mounts. A container can receive published artifacts as read-only content and caches as writable content, which aligns with least-privilege operation and limits accidental mutation.

Why the specification avoids runtime-first taxonomy

A runtime-first taxonomy looks attractive at first because operators often think in terms of engines. Over time, however, it causes duplication, ambiguity, and migration pain.

One model may be valid for several runtimes, and one runtime may support multiple artifact types. vLLM is a useful example: it has GGUF support, but the support is documented as experimental and currently constrained, which makes runtime naming a poor foundation for canonical storage design.[cite:24] Organizing the pool around the runtime would let temporary implementation details shape a durable filesystem contract.

A runtime-first tree also encourages anti-patterns such as format duplication, runtime-specific copies of the same payload, and model identity split across several unrelated directories. The specification avoids those problems by keeping artifact identity above runtime identity.

Interoperability benefits

The layout is designed to be boring in a good way. Developers can predict where canonical artifacts live, where writable caches belong, and where unfinished imports will accumulate.

That predictability helps across teams. Packagers can publish into one known tree. Runtime integrators can mount one read-only root plus one writable root. Tooling authors can build scanners, validators, and indexers against a stable contract instead of a product-specific filesystem convention.

The result is a shared vocabulary: published artifacts, cache material, and staging jobs each have a distinct place and a distinct lifecycle. Once that distinction becomes habitual, cross-project cooperation gets easier.

Design trade-offs

The layout deliberately favors correctness and operator clarity over maximum convenience.

That means some workflows are less permissive than they could be. Direct writes into the canonical model tree are forbidden, symlink publication is constrained, and imports require a staging step. Those choices add discipline, but they also reduce the chance of silent corruption or ambiguous state.

The layout also does not try to encode every detail into the path. Runtime preferences, scheduling hints, compatibility matrices, and human-facing labels belong in metadata, not in the canonical path contract. Keeping the path shape small and stable is part of what makes the specification durable.

Long-term intent

`pool_layout_v1` is designed as a lowest-common-denominator contract that can survive changes in runtimes, quantization fashions, and packaging workflows.

The long-term idea is simple: artifact identity should remain stable, publication should be trustworthy, caches should be disposable, and imports should be recoverable. As more tools adopt the same contract, the host filesystem becomes easier to reason about than ad hoc product-specific layouts.

That is the core rationale for the specification. It is not only a directory layout; it is a reliability boundary for shared local LLM infrastructure.